

Course: Image Processing 31651

Assignment #41

Noise and Median Filtration

	ID (4 last digits)	Shorten Name	Photo of the student	
Student #1	1950	shienfeld		
Student #2	2210	pony		
Student #3	7939	akimov		

Assignment #41: Noise and Median Filtration

In the file ImProcInPlainC.h defined numerical values of:
NUMBER_OF_ROWS and NUMBER_OF COLUMNS

#	Image Description
1	Google for a color photo of “sky and clouds” Convert it to a Gray image Apply to this image contrast 0.92 and Brightness +8 Store result as A411 (Hint: now image A411 contains zero number of black pixels and zero number of white pixels)
2	Spoil image A411.bmp by adding in a pseudo-random way 70 white points 40 black points and by adding 6 white vertical lines positioned in a pseudorandom way Length of the line is 30 pixels Width of the line vary in a pseudorandom way from 1 to 3 pixels (Store resulted image as A412.bmp)
3	By using Median Filtration try to “clean” image A412.bmp Use at least 3 sizes of median filter. And store resulted files as A413.bmp, A414.bmp, A415.bmp. Count black pixels and white pixels in the images A412.bmp, A413.bmp, A414.bmp, A415.bmp (Do not add into this count “black frame pixels” !!!) Explain results
	For report use YOUR OWN STYLE. From YOUR Power Point customer must understand what was done, how and what are the results (Lecturer may later ask to add slides) Done: 411, 412, 413 REMOVE ALL RED when the items are ready Important hint: Fill report pages while working. Do not complain “not enough time” in case you did not follow THIS rule.

41.1 - first important function – "Contrast_Brightness_change"

```
19 void Contrast_Brightness_change(unsigned char image[][NUMBER_OF_COLUMNS], float c, float b)
20 {
21     unsigned char* ptrToPixels = image[0];
22     for (int pixel = 0; pixel < NUMBER_OF_COLUMNS * NUMBER_OF_ROWS; pixel++, ptrToPixels++)
23         *ptrToPixels = c * (*ptrToPixels) + b; // c- contrast b -brightness
24 }
```

obtaining a pointer to the first element of the image array. This pointer (`ptrToPixels`) will be used to iterate through all the pixels in the image.

Inside the loop, each pixel value is adjusted by applying the contrast and brightness transformation:
 $*ptrToPixels = c * (*ptrToPixels) + b.$

for loop that runs through all the pixels in the image for covering every pixel in the 2D image array.

In addition : The pointer `ptrToPixels` is incremented to point to the next pixel in the image.

note 1: **c** is a float representing the contrast adjustment factor. A value greater than 1 increases contrast, while a value between 0 and 1 decreases contrast.

note 2 : **b** is a float representing the brightness adjustment factor. Adding a positive value increases brightness, while adding a negative value decreases brightness.

41.2 - second important function – “add_dots_to_pic”

```

33 void add_dots_to_pic(unsigned char image[][NUMBER_OF_COLUMNS], int num_of_dots, int color)
34 {
35     unsigned char* ptrToPixels = image[0];
36     for (int i = 0; i < num_of_dots; i++)
37     {
38         int pix_place = uniform_distribution(0, NUMBER_OF_ROWS * NUMBER_OF_COLUMNS);
39         *(ptrToPixels + pix_place) = color;
40     }
41 }

```

obtaining a pointer to the first element of the image array. This pointer (ptrToPixels) will be used to iterate through all the pixels in the image.

A for loop runs num_of_dots times, adding one dot per iteration.

The function then sets the pixel at the random position to the specified color. It calculates the address of the random pixel by adding pix_place (the random position) to the base pointer (ptrToPixels) and then computes this pointer to set the pixel value.

Inside the loop, the function calls the function **uniform_distribution** to generate a random position within the bounds of the image. This ensures that the dots are placed at random locations. (see explanation below regarding **uniform_distribution** function)

```

26 int uniform_distribution(int rangeLow, int rangeHigh) {
27     double myRand = rand() / (1.0 + RAND_MAX);
28     int range = rangeHigh - rangeLow + 1;
29     int myRand_scaled = (myRand * range) + rangeLow;
30     return myRand_scaled;
31 }

```

The uniform_distribution function generates a random integer within a specified range, which is used to determine the random positions for the dots. **The function ensures that the dots are placed at random locations within the image.**

```

114 srand(time(0));

```

Another important addition – set in the main – will be explained later

41.3 - third important function – "Add_Random_white_Lines"

5

```
43 void Add_Random_white_Lines(unsigned char image[][NUMBER_OF_COLUMNS], int number_Of_Lines, int line_length, int line_width)
44 {
45     for (int i = 0; i < number_Of_Lines; i++)
46     {
47         unsigned char* ptrToPixels = image[0];
48
49         // Calculate the valid range for pixel placement
50         int validRowStart = line_length;
51         int validRowEnd = NUMBER_OF_ROWS - line_length - 1;
52         int validColStart = 0;
53         int validColEnd = NUMBER_OF_COLUMNS - line_width - 1;
54
55         // Ensure the random position keeps the line within boundaries
56         int randomRow = uniform_distribution(validRowStart, validRowEnd);
57         int randomCol = uniform_distribution(validColStart, validColEnd);
58
59         int pix_place = randomRow * NUMBER_OF_COLUMNS + randomCol;
60         int randomWidth = uniform_distribution(1, line_width);
61
62         for (int j = 0; j < randomWidth; j++)
63         {
64             for (int k = 0; k < line_length; k++)
65             {
66                 int currentRow = randomRow - k;
67                 int currentCol = randomCol + j;
68                 if (currentRow >= 0 && currentRow < NUMBER_OF_ROWS && currentCol >= 0 && currentCol < NUMBER_OF_COLUMNS)
69                 {
70                     *(ptrToPixels + currentRow * NUMBER_OF_COLUMNS + currentCol) = 255;
71                 }
72             }
73         }
74     }
75 }
```

A for loop runs number_Of_Lines times, adding one vertical line per iteration.

the function calls **uniform_distribution** to generate a random position within the bounds of the image. This ensures that the starting position of each line is chosen randomly.

The function calls **uniform_distribution** again to determine the width of the line. This width is a random value between 1 and the specified maximum width (3).

Two nested for loops are used to set the pixel values for the line. The outer loop iterates over the width of the line, and the inner loop iterates over the length of the line.

41.4 - fourth important function – "count_num_blk_wht_pix" – part 1

6

Deeper explanation about that function in the next slide!

We decided to pick that a certain approach to ensure that the count of black pixels remains accurate, even after multiple filtering operations that might introduce unwanted edge artifacts. Will be explained deeper next slide.

The `count_num_blk_wht_pix` function counts the number of black and white pixels in a given grayscale image, while ignoring black pixels that are located on the edges of the image. This is useful when multiple filtering operations have been applied to the image, which can introduce unwanted black pixels around the edges.

```
121 void count_num_blk_wht_pix(unsigned char image[][NUMBER_OF_COLUMNS], const char name[],
122                             int filterHalfWidth, int filterHalfHeight, int numFilters)
123 {
124     int black_pix = 0;
125     int white_pix = 0;
126
127     // Calculate the effective edge thickness
128     int edgeThickness = numFilters * max(filterHalfWidth, filterHalfHeight);
129
130     unsigned char* ptrToPixels = image[0];
131     for (int row = 0; row < NUMBER_OF_ROWS; row++)
132     {
133         for (int col = 0; col < NUMBER_OF_COLUMNS; col++)
134         {
135             unsigned char pixelValue = *(ptrToPixels + row * NUMBER_OF_COLUMNS + col);
136             bool isEdgePixel = (row < edgeThickness || row >= NUMBER_OF_ROWS - edgeThickness ||
137                               col < edgeThickness || col >= NUMBER_OF_COLUMNS - edgeThickness);
138
139             if (!isEdgePixel)
140             {
141                 if (pixelValue == BLACKVAL) black_pix++;
142                 else if (pixelValue == WHITEVAL) white_pix++;
143             }
144             else if (pixelValue == WHITEVAL)
145             {
146                 white_pix++;
147             }
148         }
149     }
150     cout << "in pic: " << name << " number of black pixels is: " << black_pix;
151     cout << " number of white pixels is: " << white_pix;
152     cout << endl;
153 }
```

41.4 - fourth important function –” `count_num_blk_wht_pix`” – **part 2**

The function first calculates the effective edge thickness based on the number of filters applied and the filter dimensions.



The effective **edge thickness** is determined by multiplying the maximum of `filterHalfWidth` and `filterHalfHeight` by `numFilters`.



This value represents the number of rows and columns from the edges that should be considered as "edge pixels" and thus ignored in the black pixel count.



Inside the main for loop (line 131) , the function checks the value of each pixel:

1. If the pixel is not an edge pixel >>> If the pixel value is equal to `BLACKVAL` (which is defined as 0), it increments the `black_pix` counter.
2. If the pixel is not an edge pixel >>> If the pixel value is equal to `WHITEVAL` (which is defined as 255), it increments the `white_pix` counter.
3. If the pixel is an edge pixel >>> Only white pixels are counted by incrementing the `white_pix` counter. Black pixels in the edge region are ignored.

After the loop, the function prints the counts of black and white pixels using `cout`.
The printed message includes the name of the image and the counts of black and white pixels.

41.5-fifth important function –” DoMedianFiltration” – part 1

8

```
97
98 void DoMedianFiltration(unsigned char src[][NUMBER_OF_COLUMNS], unsigned char dest[][NUMBER_OF_COLUMNS],
99 unsigned char medianBuffer[], int filterHalfWidth, int filterHalfHeight)
100 {
101     int medianBufferSize = (2 * filterHalfWidth + 1) * (2 * filterHalfHeight + 1);
102     unsigned char* ptrToPixels = dest[0];
103     for (int i = 0; i < NUMBER_OF_ROWS * NUMBER_OF_COLUMNS; i++) { *ptrToPixels++ = 0; }
104     for (int row = filterHalfHeight; row < NUMBER_OF_ROWS - filterHalfHeight; row++)
105     {
106         for (int column = filterHalfWidth; column < NUMBER_OF_COLUMNS - filterHalfWidth; column++)
107         {
108             int counter = 0;
109             for (int y = -filterHalfHeight; y <= filterHalfHeight; y++)
110             {
111                 for (int x = -filterHalfWidth; x <= filterHalfWidth; x++)
112                 {
113                     medianBuffer[counter++] = src[row + y][column + x];
114                 }
115             }
116             dest[row][column] = MedianValue(medianBuffer, medianBufferSize);
117         }
118     }
119 }
```

Explanation of the function – next slide – because of the major importance and complication of the function we will explain a bit deeper. We will explain the code step by step / part by part .

41.5-fifth important function –” DoMedianFiltration” – **part 2**

9

```
void DoMedianFiltration(unsigned char src[][NUMBER_OF_COLUMNS], unsigned char dest[][NUMBER_OF_COLUMNS],  
                        unsigned char medianBuffer[], int filterHalfWidth, int filterHalfHeight)
```

First – we will present the inputs of the function – the filtration process is a bit more complicated so we decided to start by explaining the function step by step

src = This is a 2D array of unsigned char representing the source grayscale image to be filtered.

dst = This is a 2D array of unsigned char where the filtered image will be stored.

medianBuffer = This is an array used to store the neighborhood pixel values for calculating the median.

filterHalfWidth = This is the half-width of the filter window. The full filter width is $2 * \text{filterHalfWidth} + 1$.

filterHalfHeight = This is the half-height of the filter window. The full filter height is $2 * \text{filterHalfHeight} + 1$.

The purpose of the function is to apply a median filter to a given image.

Previously we explained about the goal of the filter – but in short : creating a new image where each pixel's value is replaced by the median value of the pixels in a neighborhood around it.

41.5-fifth important function –” DoMedianFiltration” – **part 3**

10

```
int medianBufferSize = (2 * filterHalfWidth + 1) * (2 * filterHalfHeight + 1);
```

```
for (int row = filterHalfHeight; row < NUMBER_OF_ROWS - filterHalfHeight; row++)  
{  
    for (int column = filterHalfWidth; column < NUMBER_OF_COLUMNS - filterHalfWidth; column++)  
    {
```



The medianBufferSize is calculated as the total number of pixels in the filter window which is the size of the median buffer



The outer loop iterates over the rows of the image starting from filterHalfHeight to NUMBER_OF_ROWS - filterHalfHeight.
Important to do because it helps us to avoid processing the edges of the image where the filter window would extend outside the image boundaries.
The inner loop iterates over the columns

41.5-fifth important function –” DoMedianFiltration” – **part 4**

11

```
int counter = 0;
for (int y = -filterHalfHeight; y <= filterHalfHeight; y++)
{
    for (int x = -filterHalfWidth; x <= filterHalfWidth; x++)
    {
        medianBuffer[counter++] = src[row + y][column + x];
    }
}
```

Use nested loops to iterate over the neighborhood defined by filterHalfHeight and filterHalfWidth.

Collect pixel values from the source image into medianBuffer. medianBuffer[counter++] stores the pixel value at position (row + y, column + x) from the source image.

```
dest[row][column] = MedianValue(medianBuffer, medianBufferSize);
```

Call new function - **MedianValue** - to compute the median of the pixel values stored in medianBuffer. Then - Assign the computed median value to the corresponding pixel (row, column) in the destination image.

Next , we will explain about the **MedianValue** assistance function

41.6-sixth important function – “MedianValue”

The MedianValue function computes the median of an array of pixel values.

```
79 unsigned char MedianValue(unsigned char medianBuffer[], int medianBufferSize)
80 {
81     int middle = (medianBufferSize + 1) / 2;
82     unsigned char temp;
83     for (int i = 0; i < middle; i++)
84     {
85         for (int j = 1; j < medianBufferSize - 1; j++)
86         {
87             if (medianBuffer[j] > medianBuffer[j - 1])
88             {
89                 temp = medianBuffer[j];
90                 medianBuffer[j] = medianBuffer[j - 1];
91                 medianBuffer[j - 1] = temp;
92             }
93         }
94     }
95     return medianBuffer[middle];
96 }
```

Calculate the middle index of the buffer.

Sort the buffer partially until the middle index. This is a simplified sorting algorithm that ensures the median value is in the correct position.

The function returns the value at the middle index of the partially sorted buffer as the median.

41.7-ninth important function – "main"

13

```
157 int main()
158 {
159     srand(time(0));
160
161     unsigned char buffer_3X3[9] = {};
162     unsigned char buffer_5X5[25] = {};
163     unsigned char buffer_7X7[49] = {};
164     unsigned char GrayImage[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS];
165     unsigned char dst[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS];
166
167     LoadGrayImageFromTrueColorBmpFile(GrayImage, "clouds.bmp");
168     StoreGrayImageAsGrayBmpFile(GrayImage, "clouds1.bmp"); // without contrast and brightness
169     Contrast_Brightness_change(GrayImage, 0.92, 8);
170     StoreGrayImageAsGrayBmpFile(GrayImage, "A411.bmp");
171
172     count_num_blk_wht_pix(GrayImage, "A411.bmp", 0, 0, 0); // No filters applied yet
173
174     add_dots_to_pic(GrayImage, 70, WHITEVAL);
175     add_dots_to_pic(GrayImage, 40, BLACKVAL);
176     Add_Random_white_Lines(GrayImage, 6, 30, 3);
177     StoreGrayImageAsGrayBmpFile(GrayImage, "A412.bmp");
178
179     count_num_blk_wht_pix(GrayImage, "A412.bmp", 0, 0, 0); // No filters applied yet
180
181     DoMedianFiltration(GrayImage, dst, buffer_3X3, 1, 1);
182     StoreGrayImageAsGrayBmpFile(dst, "A413.bmp");
183     count_num_blk_wht_pix(dst, "3x3 Filter", 1, 1, 1); // One 3x3 filter applied
184
185     DoMedianFiltration(GrayImage, dst, buffer_5X5, 2, 2);
186     StoreGrayImageAsGrayBmpFile(dst, "A414.bmp");
187     count_num_blk_wht_pix(dst, "5x5 Filter", 2, 2, 2); // Two filters applied
188
189     DoMedianFiltration(GrayImage, dst, buffer_7X7, 3, 3);
190     StoreGrayImageAsGrayBmpFile(dst, "A415.bmp");
191     count_num_blk_wht_pix(dst, "7x7 Filter", 3, 3, 3); // Three filters applied
192
193     WaitForUserPressKey();
194     return 0;
195 }
```

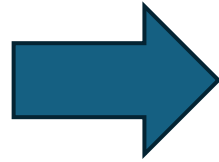
These arrays are initialized to hold pixel values for the images and the median filter buffers: `buffer_3X3`, `buffer_5X5`, and `buffer_7X7` are used to store pixel values for median filtering with different filter sizes, `GrayImage` stores the grayscale image, `dst` stores the resulting image after applying median filtering.

`srand(time(0))` initializes the random number generator with the current time. This ensures that the sequence of random numbers generated in the program will be **different** each time it runs.

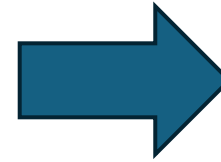
The `count_num_blk_wht_pix` function is called after each major image processing step. Pay attention to its parameters considering the applied NXN median filter

41.8-presentation of the created images by the program – **part 1**

Clouds.bmp



Clouds1.bmp



A411.bmp



Continue of the
created images
next slide



41.8-presentation of the created images by the program – **part 3**

15

A411.bmp



A412.bmp



A413.bmp



A414.bmp

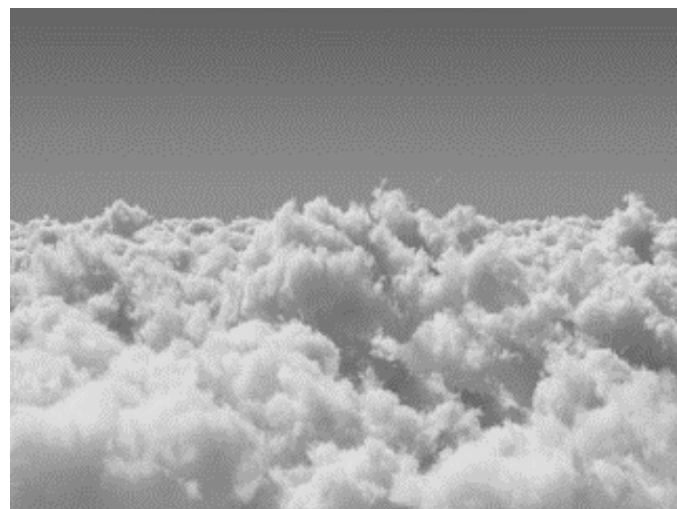


A415.bmp



41.8-presentation of the created images by the program – **part 4**

Clouds1.bmp



Point:	Mouse	Expl.	(M-E)
Row	76	76	0
Column	81	81	0
Red	209	209	0
Green	209	209	0
Blue	209	209	0



A411.bmp

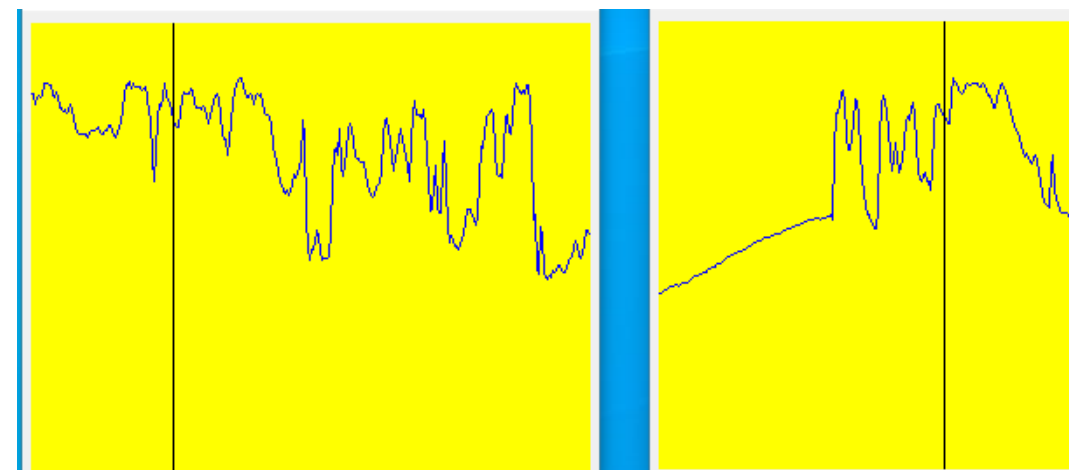
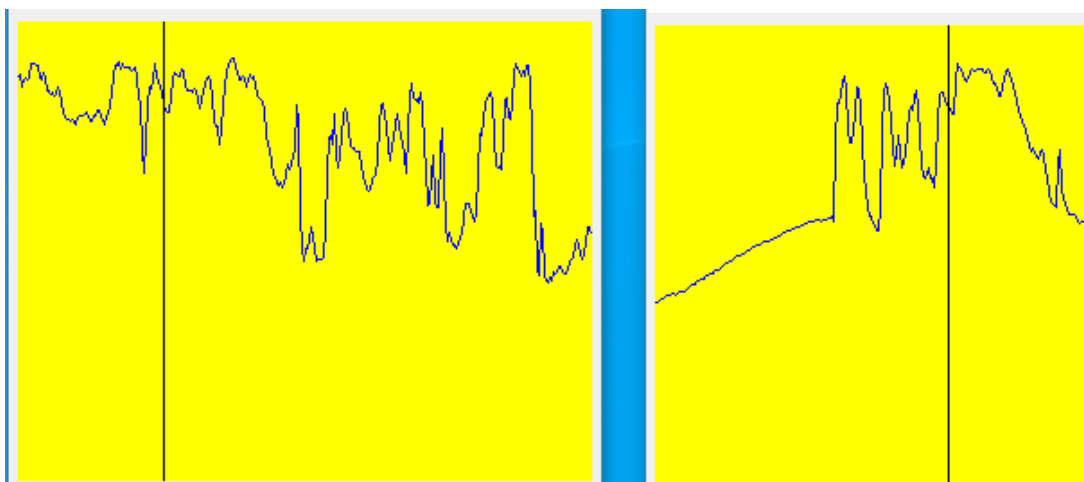


Point:	Mouse	Expl.	(M-E)
Row	76	76	0
Column	81	81	0
Red	200	200	0
Green	200	200	0
Blue	200	200	0



Clouds1.bmp is the grayscale version of the original clouds.bmp color image

A411.bmp is an improved version of clouds1.bmp – improvement in contrast and brightness – in the point explorer one can see improvement from 209 to 200.



41.8-presentation of the created images by the program – **part 5**

17

First and foremost - for verification purposes – we made sure that A411.bmp does not contain white / black pixels that we might count twice on the next stage

```
in pic: A411.bmp number of black pixels is: 0 number of white pixels is: 0  
Gray BMP File
```

A412.bmp

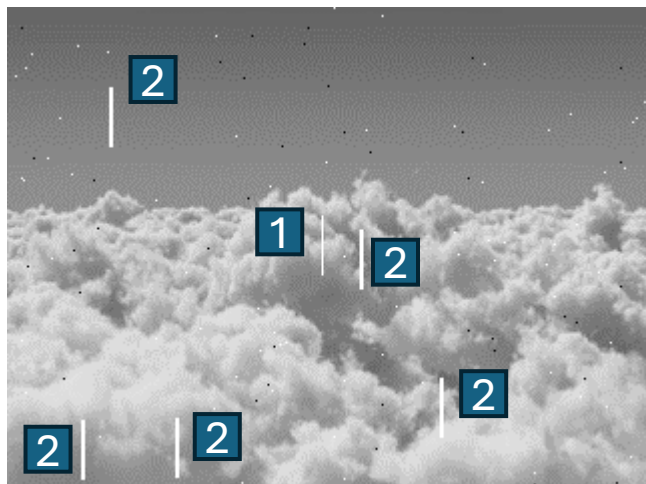


Then, We have added 6 white vertical lines positioned in random locations , the length of the lines is 30 pixels, and the width is between 1 to 3 pixels randomly.
The number of white pixels we have: 70
The number of black pixels we have: 40

```
in pic: A412.bmp number of black pixels is: 40 number of white pixels is: 400
```

The number of white pixels is **higher than 70** because of the 6 white lines in the picture. (these lines contain a lot of white pixels – a number to be calculated): five lines are 2 pixels wide, one is 1 pixels. (for the specific case we generated)
The length is 30 for all – the calculation is:
 $5 * 2 * 30 + 1 * 1 * 30 = 330 \rightarrow 400 - 330 = 70 \text{ white dots !}$

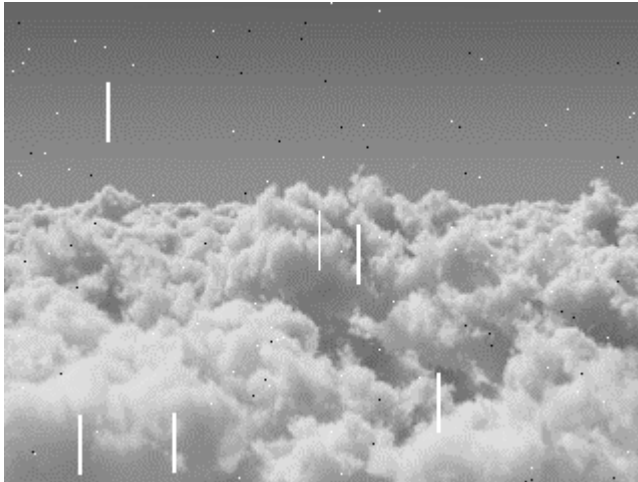
Width of pixels for white lines – checked using BMP views experimentally



41.8-presentation of the created images by the program – **part 6**

18

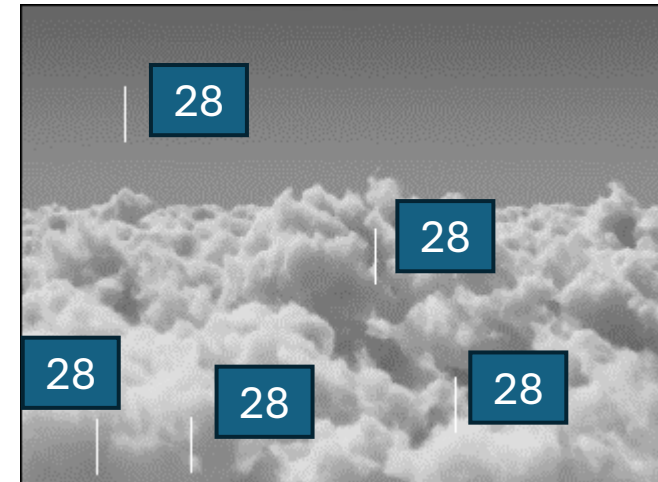
A412.bmp



New lengths of the remaining white noise is specified for further calculation

Apply 3x3 median filter

A413.bmp



In this image we can see that after implementing the median filter 3X3 on the image we succeeded to remove all the black and white points and some of the white lines. (and the remaining lines left some of their width and length)

The amount of black pixels is zero thanks to the verification process of the counting function (the black lines on the edges of the image are not taken under account in the counting process)

We can also see that a median filter 3X3 is not enough for cleaning the white lines with a width of 1 pixel.

Amount of white pixels: 140 (only the remaining white vertical lines – calculation : $5 \times 1 \times 30 = 150 - 2 - 2 - 2 - 2 - 2 = 140$)

Amount of black pixel: 0

```
in pic: 3x3 Filter number of black pixels is: 0 number of white pixels is: 140
```

41.8-presentation of the created images by the program – **part 7**

19

A412.bmp



Apply 5x5 median filter

A414.bmp



In this image we can see that after implementing a 5X5 median filter on the original image, we succeeded to clean all white and black points and also succeeded cleaning all white lines.

The median filter 5X5 cleaned the image very well and it is the best filter we tried to implement.

The main drawback is the decrease in quality and sharpness of the image!

Number of white pixels: 0

Number of black pixels: 0

Pay attention that we present only one tryout of the random process – in other cases the 5x5 median filter failed to filter all the white lines (few remained but smaller and thinner).

```
in pic: 5x5 Filter number of black pixels is: 0 number of white pixels is: 0
```

41.8-presentation of the created images by the program – part 8

20

A412.bmp



Apply 7x7 median filter

A415.bmp



In this image we can see that after implementing a 7X7 median filter on the image we succeeded to clean all white and black points and succeeded to clean all the white lines!

This median filter succeeded to filter all the salt & paper noises and the white lines, but the image looks grain and not very realistic (we lost the sharpness of the original image). (even worse than after applying 5x5 filter)

We can also see that the frame around the image is very large in compared to the previous median filters.

Important to know is that in every tryout we did – the 7x7 succeeded to clean all the noise (compared to the 5x5 that sometimes succeed but sometimes fails. The prize of the full cleaning is the deprivation in the quality of the image!

Number of white pixels: 0

Number of black pixels: 0

```
in pic: 7x7 Filter number of black pixels is: 0 number of white pixels is: 0
```

41.9-what did we learn?

1. We explored the differences between filtering with small and large matrices (buffers), noting that smaller buffers preserve more detail, while larger buffers provide stronger noise reduction at the cost of blurring.
2. We learned how to implement a median filter with various buffer sizes, adjusting to different image processing needs.
3. We identified when a specific filter size is appropriate for an image, depending on the type and extent of noise.
4. We discovered that the buffer size should correspond to the noise size—small buffers handle minor noise effectively, while larger buffers are necessary for more significant artifacts like lines, with a trade-off between noise reduction and image sharpness.